

Angular (v20+) Hands-On Lab Report

Incremental Project: Student Course Portal

Hands-On:	Hands-On 9 [Advanced-Professional]
Topic:	NgRx Store, Reducers, Effects, Selectors, Actions & DevTools
Developer:	Cathy George
OS Version:	Windows 11
Date:	July 22, 2026
Track:	Java FSE Angular Track

1. Objective

The primary objective of this exercise is to implement reactive state management inside the Student Course Portal using the NgRx framework. By establishing a dedicated 'Enrollment Store', we abstract enrollment tracking away from ad-hoc component variables. We introduce structured Actions representing state events, pure Reducer functions to handle state modifications synchronously, asynchronous Effects to integrate server catalog patches, and memoized Selectors to retrieve optimized state slices. Lastly, we register Redux DevTools for real-time history debugging.

2. Angular Concepts Covered

- **NgRx Store:** Redux-inspired global state container providing single-source-of-truth reactivity.
- **Actions (createAction):** Strongly typed payloads representing unique events that trigger state updates.
- **Reducers (createReducer):** Pure functions taking current state and actions, and returning new immutable state.
- **Effects (createEffect):** Side-effect handlers mapping actions to HTTP network payloads and dispatching result states.
- **Selectors (createSelector):** High-performance memoized queries yielding specific state properties.
- **DevTools (provideStoreDevtools):** Integrating Redux DevTools Extension to inspect state transitions and trigger time-travel logs.

3. NgRx Store Configuration & State Definition

Enrollment Actions (src/app/store/enrollment/enrollment.actions.ts)

```
export const loadEnrollments = createAction('[Enrollment] Load Enrollments');
export const loadEnrollmentsSuccess = createAction(
  '[Enrollment] Load Enrollments Success', props<{ enrolledIds: number[] }>()
);
export const enrollCourse = createAction(
  '[Enrollment] Enroll Course', props<{ courseId: number; courseCode: string }>()
);
export const enrollCourseSuccess = createAction(
  '[Enrollment] Enroll Course Success', props<{ courseId: number; courseCode: string }>()
);
```

Enrollment Reducer (src/app/store/enrollment/enrollment.reducer.ts)

```
export const enrollmentReducer = createReducer(
  initialState,
  on(EnrollmentActions.loadEnrollmentsSuccess, (state, { enrolledIds }) => ({
    ...state, enrolledCourseIds: enrolledIds, loading: false
  })),
  on(EnrollmentActions.enrollCourseSuccess, (state, { courseId }) => ({
    ...state,
    enrolledCourseIds: state.enrolledCourseIds.includes(courseId)
      ? state.enrolledCourseIds : [...state.enrolledCourseIds, courseId],
    loading: false
  }))
);
```

4. NgRx Effects & Service Binding

Asynchronous Side-Effects (src/app/store/enrollment/enrollment.effects.ts)

```
enrollCourse$ = createEffect(() =>
  this.actions$.pipe(
    ofType(EnrollmentActions.enrollCourse),
    switchMap(action =>
      this.courseService.updateCourseEnrollment(action.courseId, true).pipe(
        map(() => EnrollmentActions.enrollCourseSuccess({
          courseId: action.courseId, courseCode: action.courseCode
        })),
        catchError(error => of(EnrollmentActions.enrollCourseFailure({ error: error.message })))
      )
    )
  );
```

Refactored Enrollment Service (src/app/services/enrollment.service.ts)

```
export class EnrollmentService {
  private store = inject(Store);
  private courseService = inject(CourseService);

  getEnrolledCourseIds(): Observable<Set<number>> {
    return this.store.select(selectEnrolledCourseIds).pipe(
      map(ids => new Set(ids))
    );
  }

  enrollCourse(course: Course): void {
    this.store.dispatch(EnrollmentActions.enrollCourse({
      courseId: course.id, courseCode: course.code
    }));
  }
}
```

5. Verification and Store Debugging

The enrollment store operations and state travels were successfully verified:

[Screenshot Placeholder]: NgRx Initial State Load - Redux DevTools displays [Enrollment] Load Enrollments action followed by [Enrollment] Load Enrollments Success, initializing active course enrollment lists.

[Screenshot Placeholder]: Enroll Action Dispatch - Clicking 'Enroll' on a course card triggers the [Enrollment] Enroll Course action, displaying action properties and updating state asynchronously.

[Screenshot Placeholder]: State Diff Inspection - Developer tools panel highlights the state array changes (e.g. adding ID 1 to enrolledCourseIds) in a highlighted green difference tree.

[Screenshot Placeholder]: Redux Time Travel Playback - Running the devtools timeline slider dynamically rewinds enrollment states, recalculating credits count reactive labels instantly.

6. Learning Outcomes & Conclusion

- ✓ Constructed structured action schemas for modular, scaleable states.
- ✓ Implemented immutable array expansions and filter operations inside pure Reducers.
- ✓ Created side-effect handlers to capture network requests asynchronously.
- ✓ Used memoized selectors to map course items and compute aggregated credit values.
- ✓ Integrated Redux DevTools providers to enable visual history tracking.

Conclusion

Hands-On 9 successfully migrates dynamic enrollment tracking in the Student Course Portal to an enterprise-grade NgRx store. The implementation fulfills standard declarative architecture guidelines, providing centralized state control and visual time-travel logging.